

EuroFinder

IDEA PRINCIPAL

Eurofinder nace de la idea de crear un directorio de todas las plataformas p2p europeas (Wallapop.es , Leboncoin.fr , Markplaats.nl, jofogas.hu...) , para encontrar un producto específico al mejor precio y mejor coincidencia con tu búsqueda ampliando la barrera de posibilidad a 27 países que forma la UE y no solo tu propio país.

Nota: Eurofinder NO es un Marketplace, es un directorio que recopila los enlaces de los Marketplace originales.

Lo que busco abarcar con EuroFinder vendría a buscar la democratización de la compra y venta entre particulares de la Unión Europea, como si todo fuera un mismo país , de la misma manera que en estados unidos tienen eBay pues aquí sería un equivalente pero en modo directorio y no plataforma de negociación directamente, es decir, que una persona en Hungría o Croacia pueda encontrar un artículo exclusivo , difícil , que no se fabrica ya, o que simplemente es demasiado caro en su país , vendiéndose muy barato en la principal página p2p de Francia , Alemania , Lituania. O cualquier país de la UE

Uno nunca sabe si lo que para ti no vale tanto, a lo mejor otro no logra encontrarlo en ningún lado en su país, ni nuevo ni de segunda mano...

...Uno no sabe si hay una persona vendiendo su coche de manera desesperada tirada de precio o porque ese coche ya no está de moda en su país por un precio mucho mas bajo que en Alemania , no tiene por qué ser siempre en Alemania que se encuentran coches baratos ni siempre en Italia que se encuentran las prendas de lujo exclusivas y al mejor precio.

Así que aquí entraría la ventaja.

La UE nos ofrece libre comercio de bienes y servicios entre países miembros entonces ..porque no aprovechar la ventaja? Según las leyes es igual de fácil comprar, importar un producto de Francia, Portugal que de un país miembro de la UE en la otra punta del continente.

EuroFinder: Proyecto académico

En esta primera versión de EuroFinder, que esta hecha expresamente para demostrar que he asimilado los conceptos del Master, he decidido crear lo que viene a ser una prueba de concepto (PoC), simplemente crear un metabuscador funcional, con un mínimo de funciones simplemente para demostrar que es viable y útil, para en un futuro convencer a inversiones y a las plataformas propietarias de la información que inviertan y me den acceso a sus datos para poder compartirlos en mi directorio de manera legal y segura.

Proyecto: Crear y dar vida al PoC con herramientas de IA

En este proyecto digo que CREO el PoC con IA porque voy a utilizar herramientas de generación de código con lenguaje natural (Vibe coding) como Claude Code y Codex para generar el frontend y el backend del servicio.

Y digo que DOY VIDA al PoC con IA porque la lógica del funcionamiento usa el framework de Langgraph con múltiples nodos que combinan scrapers y agentes de IA inteligentes que entienden, analizan, preparan, buscan, normalizan y presentan el resultado esperado por el usuario según la información que aporte en un principio en el input.

Para este primer contacto con el proyecto me he propuesto crear un simple PoC, que simplemente funcione y cumpla con el pipeline completo, sin indagar mucho ni buscar demostrar ninguna capacidad de beneficio económico, solo demostrar que puedo crear una solución utilizando todas las herramientas necesarias de IA.

Parte 1: Crear la solución con vibe coding

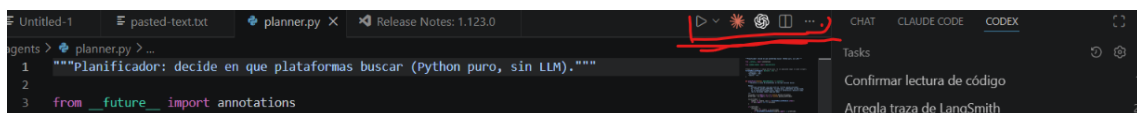
Me he decantado por crear el código con el framework de langgraph , y usar técnicas de vibe coding para crear el stack completo de la aplicación. Vibe coding no viene a ser mas que dar instrucciones en lenguaje natural a llms que son buenas generando código así vas ajustando el código mientras lo pruebas y le vas dando tu feedback al modelo para que ajuste a tu gusto las diferentes partes de tu aplicación. Pero no es tan fácil como darle al modelo tu idea y dejarle hacer , hay varios parámetros que se tienen que tener en cuenta , para mi proyecto he usado tecnología de Open AI y Anthropic (ChatGPT, Codex) (Claude.ai , Claude code , Claude design)

Mayoritariamente con Anthropic así que voy a hablar de Claude mas que nada

Los puntos que hay que tener en cuenta para hacer vibe coding correctamente son :

- El modelo que vas a usar y el costo de los tokens de entrada y salida, Yo he elegido el modelo Opus 4.7 y Opus 4.8 ya que son los motores mas fuertes para generar el código a dia de hoy (Ojo que codex con gpt 5.5 se ha puesto al mismo nivel casi) . Al principio puedes tirar del saldo de tu cuenta de Claude.ai pero se acaban rápido y el cuello de botella de la tecnología de anthropic es que no tienen suficiente computo como para poder ser generosos en las suscripciones así que hay que tirar de la API de la plataforma de claude y hay que pagar tokens que suben hasta la friolera de 25\$ por millón de token de salida.

- Elegir el entorno en el que vas a trabajar, usando la extension de Claude code en un IDE o directamente usar la app de Claude code sin ver el código como se va escribiendo y no poder interactuar con el código. En mi caso he usado Vs Code + la extensión de Claude Code y la de codex.



- Antes de empezar a generar el código en el entorno elegido, el hacer un brainstorming y una planificación adecuada explicándole tu idea a modelos de razonamiento, en mi caso he usado Claude.ai con el modelo opus 4.7, le he explicado mi idea de principio a fin , me ha ayudado a pensar mejores detalles y le he pedido que me cree un documento markdown (claude.md) con la idea explicada a claude code y para mi los prompts que le tengo que dar para estructurar bien la generación del código y no se ponga a hacer todo de golpe y mal, de manera que si se rompe algo siempre tienes la opción de deshacer el prompt con el botón de rewind y volver al paso de antes cuando funcionaba todo y volver a replantear el siguiente paso para que no se rompa nada.

- Resumiendo el punto anterior, dividir el proyecto en fases e ir dándole las instrucciones una a una para que el modelo haga todo de manera estructurada ya que la manera correcta es ir dándole instrucciones y feedback continuo de cada bloque.

- Si quieres añadir algo al código que no has planeado con claude.ai , lo suyo es usar el modo plan durante la sesión antes de pedirle que cambie nada , así solo planea y no ejecuta hasta que le des el visto bueno .

- Como he dicho ante hay que optimizar el uso de tokens , jugando con el modelo utilizado y el esfuerzo que hace para cada tarea , para un arreglo simple lo suyo es usar modelos baratos y rápidos como sonnet o haiku , para cosas más complejas ya tirar mejor del modelo mas inteligente y caro. Para el resto es ir jugando con mas o menos esfuerzo, con o sin razonamiento ...etc. Yo la mayor parte la he hecho con Opus 4.7 con el effort al máximo y el thinking activado buscando el ir más rápido y eso ha sido un grave error que me ha costado 120\$ en tokens pero luego he recapacitado y he ido optimizando el uso usando sonnet 4.6 y opus 4.7 solo para cuando se estancaba debia planificar algo. Me he podido ahorrar bastante dinero asi ya que solo tiraba de crédito de la api y reservaba los créditos de la suscripción para poder seguir hablando con claude.ai y no encontrarme con el bloqueo de las 5 horas.

- Una vez vaya cogiendo color el proyecto le pides a claude code que te abra un local host para que puedas correr de manera local la aplicación y ver que va fallando y que funciona bien. En mi caso a la que ya el buscador se lograba conectar con las plataformas ya he ido probando como funciona y he ido ajustando los errores tanto en el front como backend.

-Como he dicho antes se me había ido de las manos el consumo de tokens y cuando ya me vi que me había dejado demasiado dinero en claude code, hice una copia del código y le pase toda esa copia a codex con gpt 5.5 para que siga con el trabajo y sorprendentemente, me ha resuelto problemas que opus 4.7 no era capaz de resolverme (por ejemplo el conectar con la api de LangSmith) , con opus me estuve rato hasta que me harte de pagar y gpt me lo soluciono con la primera iteración asi que acabe el proyecto en esta copia que hice para codex. Y lo mejor de todo -gratis- ya que tiraba de la subscripcion de gpt plus.

-Al final de todo ya con la pagina funcional, le pase el .zip con el código a claude design y le dije que me hiciera un restyling del aspecto de la página ya que se veía muy soso, me devolvió unos cuantos archivos de código que le pase a codex y le pedí que me replicara este diseño y lo reemplace por el actual y lo acabo haciendo genial.

- Y bueno estoy dejando estar bastantes mas detalles que uno puede consultar con claude o gpt para indagar mas pero este ha sido el desarrollo de 0 a una aplicación funcional usando simple lenguaje natural.

Parte 2 , dar vida al código creado con agentes de IA

En una primera intención quise que el pipeline del PoC fuera tal que asi

QUERY (lenguaje natural)



EXTRACTOR (LLM) → categoría + filtros estructurados (Pydantic)



PLANIFICADOR (LLM) → qué plataformas + traduce query a cada idioma



SCRAPING PARALELO — escalado por dureza:

Tier 1 httpx (HTML estático)

Tier 2 Playwright (JS)

Tier 3 Playwright + stealth + proxies

Tier 4 solver de captcha (IA)

Tier 5 agente navegador con visión



SUPERVISOR (LLM) → detecta fallos, reintenta, reformula,
auto-cura selectores rotos



NORMALIZADOR → unifica formato, divisas, deduplica, separa piezas



TRADUCTOR (LLM, batch) → todo al idioma del usuario



RANKER (IA) → embeddings + precio + match + geografía → orden



ENRIQUECIMIENTO → informar de coste de envío + avisar de chollo/estafa



PRESENTADOR (LLM) → tarjetas + recomendación justificada



SALIDA → directorio rankeado + link al anuncio original

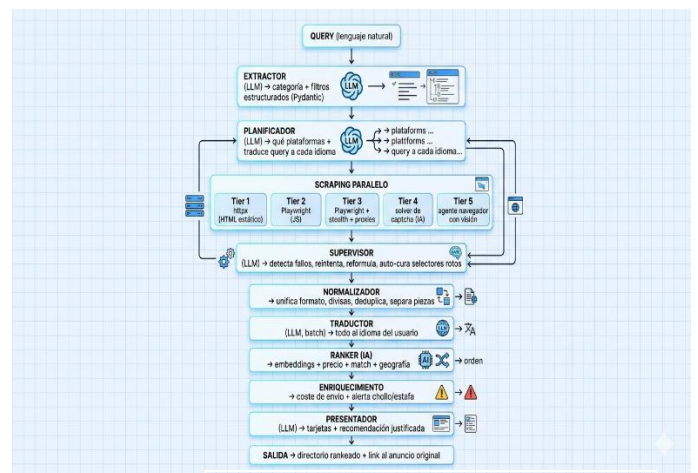


Imagen del pipeline generada con Gemini 3.5 flash

Pero por no querer complicar un proyecto académico he preferido hacer una versión mínima que solo scrapee 3 marketplaces en lugar de 27 y cumpla con estas funciones mas simples:

QUERY (lenguaje natural, español)



EXTRACTOR (LLM · Haiku) → categoría + filtros estructurados (Pydantic)



APLICAR OVERRIDES → filtros manuales del usuario (si los hay)



PLANIFICADOR → selecciona plataformas



SCRAPING PARALELO (asyncio) — Tier 1 httpx + selectolax:

Leboncoin FR - Willhaben AT -Subito IT



NORMALIZADOR → unifica formato + divisas a EUR



FILTRO → descarta ruido (precio 0, marca/modelo ausente)



TRADUCTOR (LLM · Haiku, batch) → todo al español

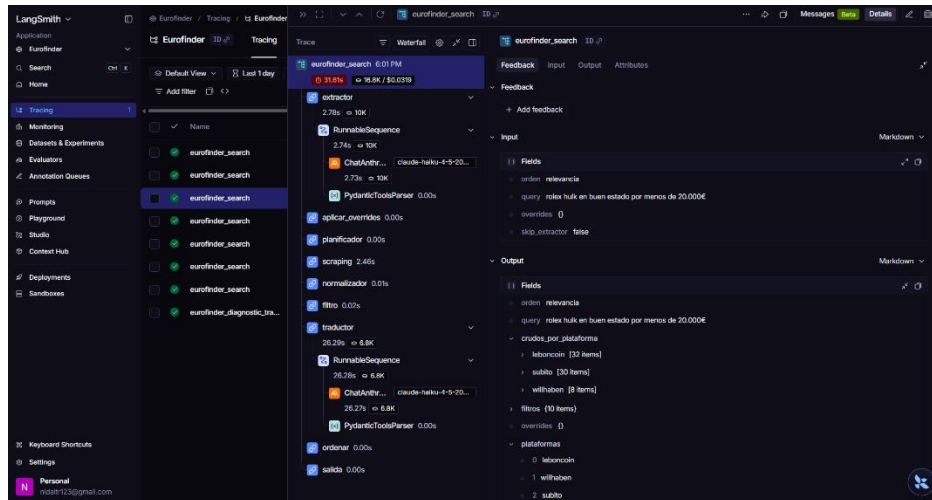


ORDENAR → por relevancia nativa de cada plataforma



SALIDA → directorio + link al anuncio original

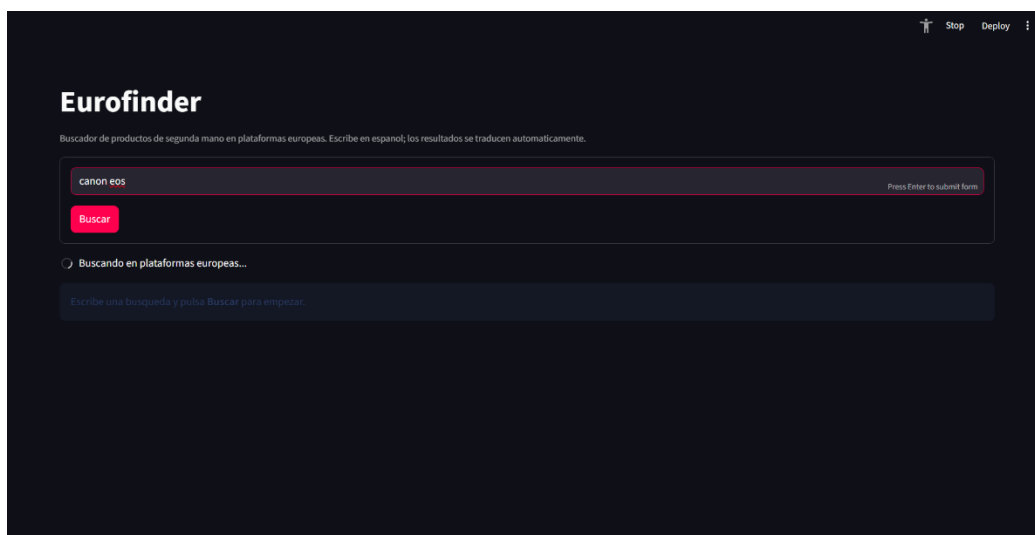
Con langsmith podemos ver el pipeline en funcionamiento:



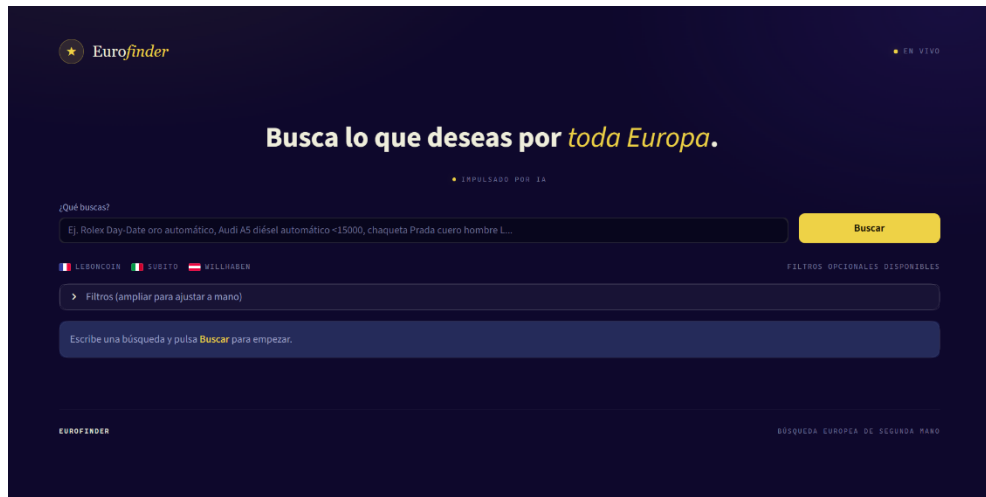
Al haber tenido un gasto económico excesivo con los tokens de opus 4.7 y opus 4.8 he optado por haiku en el extractor, sino hubiese preferido una llm mas potente para la asimilación de la query.

Como podemos ver las 3 paginas que he puesto son de tier 1 de scraper , no ha hecho falta el agente IA con visión.

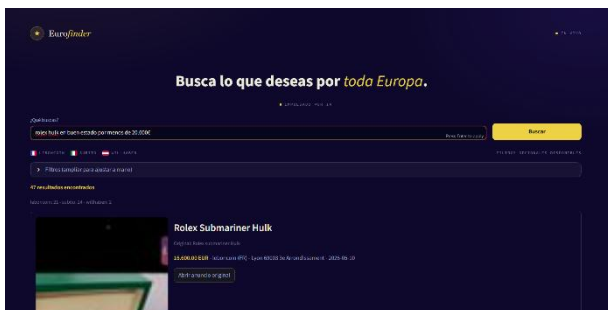
No me he querido molestar con el frontend ni el diseño de momento , he editado el código con Claude design para que pase de esto :



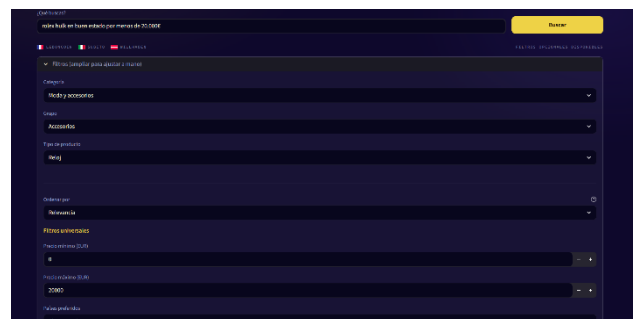
A esto:



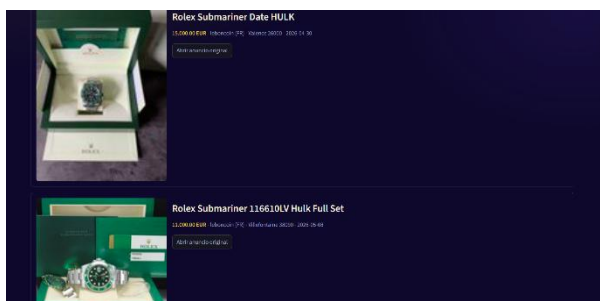
Y aquí capturas de búsquedas y resultados



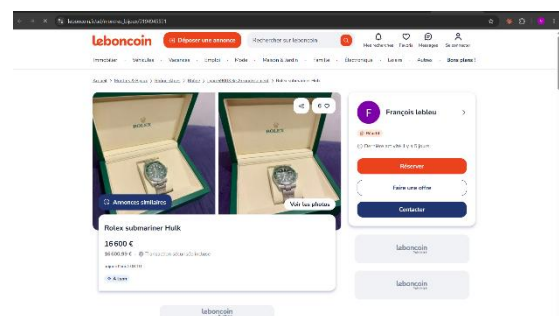
Query en lenguaje natural o opción de poner filtros manual



Ilm que entiende bien los filtros y los pone



Resultados



Anuncio original

Medidas de seguridad.

Como he dicho , es un PoC así que no he buscado añadir ninguna base de datos ni login ni pasarela de pago así que no hay mucho que poder sacar de la página , pero sí es importante proteger las credenciales de las llms y evitar inyecciones a la llm por eso que :

Entre la entrada del usuario y el extractor he añadido una función de saneado de la query. La función limita la búsqueda a 200 caracteres, elimina caracteres no imprimibles y normaliza el texto colapsando espacios múltiples, saltos de línea o tabulaciones. Mas que nda para reducir el riesgo de entradas malformadas, demasiado largas o con caracteres extraños antes de enviarlas al LLM y al resto del flujo. No es una protección completa contra todo tipo de inyección, pero sí una primera capa de seguridad dentro del sistema.

He aislado las Api keys tanto de langsmith como de Anthropic en un fichero aparte .env y he añadido ese fichero al .gitignore para que no se suba al control de versiones. Así evito el la causa mas común en el filtrado de Api keys , que es el subirlas sin querer a un repositorio.

Si se quisiera sacar a producción el proyecto en un hipotético caso tal cual está ahora, se deberían tener en cuenta al menos estos riesgos y posibles errores/ataques:

-Inyecciones de prompt:

- alguien escribe en la query instrucciones para manipular al LLM "ignora tus instrucciones y haz X".

- Solución: saneado de entrada longitud máxima, caracteres imprimibles, y que el extractor use structured output con Pydantic, que limita la salida a un schema fijo y reduce el margen de que el modelo obedezca instrucciones inyectadas.

Como trabajo futuro: un filtro previo que detecte patrones de inyección.

-Fuga de credenciales

- que las claves de Anthropic o LangSmith acaben expuestas en el código o en un repositorio público

- Solución: claves aisladas en .env y ese fichero quitado del control de versiones con .gitignore. Como trabajo futuro: gestor de secretos dedicado y rotación periódica de claves.

Coste descontrolado de tokens :

- uso masivo o malicioso que dispara el gasto de la API.
 - Solucion: límite de gasto configurado en la consola del proveedor, uso de Haiku y traducción en batch para minimizar llamadas.
- Como trabajo futuro: rate limiting por usuario y caché de búsquedas frecuentes.

Alucinación o error del LLM en la extracción:

- Riesgo: el modelo interpreta mal la query y extrae filtros incorrectos.
 - Solucion: structured output con Pydantic que valida los tipos, y sanitización post-LLM que descarta campos que no aplican al tipo de producto.
- Como trabajo futuro: evaluación continua con LLM como juez para medir la tasa de error.

Caída de un servicio externo

- Riesgo: si la API de Anthropic o una plataforma están caídas, el sistema falla.
 - Solucion: manejo de errores por nodo y aislamiento de fallos (que una plataforma caída no tumba la búsqueda).
- Como trabajo futuro: fallback a modelo alternativo.

Legalidad y conclusión final

Como he dicho antes , si se sacara a producción tendría en cuenta estos aspectos.

Ya que para mi el mayor riesgo de Eurofinder no es técnico, sino legal...

Scraping de las plataformas

Para mi es el punto mas delicado , Hacer scraping suele ir en contra de sus términos de servicio. No es del todo ilegal, pero expone a bloqueos, a obligaciones para parar la actividad o incluso a acciones legales según el país. Ya que esto es un proyecto académico , sin fin comercial ni un uso publico, el riesgo actual es bajo. Pero la solución real de cara a producción no es ocultar mejor el scraping, sino sustituirlo por acceso oficial a los datos mediante acuerdos con las plataformas

Los derechos sobre el contenido de los anuncios.

Los textos y las imágenes tienen titular de derechos. Reproducirlos o almacenarlos puede entrar en conflicto con la propiedad intelectual, y en la UE existe además una protección específica sobre las bases de datos, pensada justo contra la extracción masiva de información. Mi sistema no guarda ni republica anuncios: solo la muestra de forma temporal y redirige al original, lo que reduce ese riesgo. En producción sería necesaria una licencia o un acuerdo con la plataforma

La protección de datos personales

Los anuncios pueden contener nombres, teléfono o ubicación del vendedor, y tratar esos datos sin una base legal entra en conflicto con el RGPD, que en la UE conlleva sanciones importantes. La forma en que lo evito es no almacenar datos personales y redirigir siempre al anuncio original. Una versión productiva necesitaría una política de privacidad y un cumplimiento formal del RGPD

La responsabilidad ante posibles estafas

Si un usuario llega a un anuncio a través del directorio y resulta ser una estafa, podría plantearse hasta qué punto soy responsable. Para cortarlo, dejo claro que Eurofinder es un buscador, no un intermediario de la transacción ni responsable de los anuncios: la negociación y el pago ocurren fuera, en la plataforma original. Esto quedaría recogido en los términos de uso y, de paso, refuerza la idea de que se trata de un directorio y no de una plataforma de compraventa

La cuestión de la competencia con las otras plataformas

Al agregar sus anuncios, resto usuarios a las plataformas. En realidad es lo contrario: al redirigir al anuncio original, les apporto tráfico. Ese es precisamente mi argumento para negociar con ellas en lugar de competir, porque les llevo compradores de otros países a los que no llegaban por sí solas

La importación de los productos y la fiscalidad

La compra entre países de la UE implica cuestiones de IVA y, en productos como los vehículos, trámites de matriculación y impuestos. En en PoC no me afecta, ya que no gestiono ninguna transacción. En producción sería necesario informar al usuario de estos costes, sin asumir yo la responsabilidad fiscal

-En resumen, lo más arriesgado de Eurofinder no es el código, sino la dimensión legal. El scraping se mueve en una zona gris de términos de servicio, derechos sobre el contenido y protección de datos. Por eso lo he planteado como un buscador que redirige al anuncio original, sin almacenar, intermediar ni gestionar pagos, y por eso la evolución del proyecto no es ampliar el scraping, sino sustituirlo por acuerdos oficiales con las plataformas. El scraping me sirve para demostrar que la idea funciona, el camino serio hacia producción es convertir a esas plataformas en socias, aprovechando que les apporto tráfico internacional en lugar de competir con ellas.

<https://github.com/nidaltr/Euro-finder-Thebridge-project.git>

Atentamente , Nidal alias Nr7.00